

Amazon Clone — خطة التنفيذ

الترتيب الكامل للمشروع من الصفر للنشر

JWT Auth

React

MongoDB

Express (MVC)

💡 **القاعدة الذهبية:** نبني الباك-إند الأول (API شغال ومتجرب) ← بعدين الفرونت-إند يستهلكه. كل مرحلة تخلص وتتجرب قبل ما ننتقل للي بعدها.

هيكل المشروع (Folder Structure)

```
amazonClone/
├── backend/
│   ├── config/db.js           # MongoDB الاتصال بـ
│   ├── models/               # User, Category, Product, Order
│   ├── controllers/          # المنطق (Logic)
│   ├── routes/               # المسارات (Endpoints)
│   ├── middleware/           # auth + error
│   ├── utils/                # generateToken, seeder
│   ├── .env
│   ├── server.js             # نقطة البداية
│   └── package.json
└── frontend/                 # React + Vite
    └── src/
        ├── api/axios.js
        ├── context/          # AuthContext, CartContext
        ├── components/       # Header, Footer, ProductCard...
        ├── pages/            # Home, Product, Cart, Checkout, Login...
        ├── App.jsx
        └── main.jsx
```

المتطلبات قبل ما نبدأ 

Node.js (نسخة +18) — تتأكد بـ `node -v` ☐

MongoDB — محلي على الجهاز، أو MongoDB Atlas سحابي مجاني (الأسهل) ☐

محزّر أكواد — VS Code ☒

0 التجهيز (Setup)

الهدف: سيرفر فاضي بيشتغل + اتصال بقاعدة البيانات.

إنشاء فولدر `backend` و `npm init` ☐

تنصيب: `express mongoose dotenv cors bcryptjs jsonwebtoken` ☐

أدوات التطوير: `nodemon` ☐

ملف `env.` فيه `PORT` و `MONGO_URI` و `JWT_SECRET` ☐

`config/db.js` + `server.js` ☐

التجربة: السيرفر يشتغل و "MongoDB Connected" تظهر. ☒

1 الموديلز والميدل-وير

الهدف: أساس ال MVC جاهز.

`User.js` (مع تشفير الباسورد `bcrypt`) ☐

`Category.js` · `Product.js` · `Order.js` ☐

`errorMiddleware.js` + `authMiddleware.js` (protect + admin) ☐

`utils/generateToken.js` ☐

2 التسجيل والدخول (Auth)

الهدف: المستخدم يسجل ويدخل وياخذ توكن.

`authController` → register / login / getProfile ☐

`.../ authRoutes` → `/api/auth` ☐

التجربة: بتسجل يوزر وتأخذ JWT عن طريق Postman / Thunder Client ☒

3 المنتجات والتصنيفات (Catalog)

الهدف: قلب المتجر.

`categoryController` + `categoryRoutes` ☐

`productController` : عرض + بحث/فلتر + إدارة (أدمن) ☐

`utils/seeder.js` — تصنيفات ومنتجات تجريبية + أدمن ☐

التجربة: `GET /api/products` يرجع منتجات. ☒

4 أساس الفرونت-إند (React)

الهدف: تطبيق React بيشتغل ومربوط بال API.

إنشاء `frontend` بـ `react-router-dom` `axios` + Vite ☐

`Header/Footer` + الراوتنج + `api/axios.js` ☐

`AuthContext` ☐

التجربة: الصفحة الرئيسية بتفتح. ☒

5 عرض المنتجات (Browsing)

صفحة Home — شبكة المنتجات (`ProductCard`) ☐

صفحة تفاصيل المنتج ☐

☐ بحث + فلترة بالتصنيف/السعر

☒ التجربة: بتشوف منتجات الداتابيز على الموقع.

6 السلة (Cart)

☐ `CartContext` (localStorage)

☐ إضافة / تعديل كمية / حذف + إجمالي السعر

☒ التجربة: بتضيف منتج وتلاقيه في السلة.

7 الطلب والدفع (Checkout)

☐ صفحات Login / Register في الفرونت

☐ حماية ال Checkout + فورم العنوان + ملخص الطلب

☐ `orderController` + `orderRoutes` (إنشاء طلب + طلباتي)

☐ صفحة "تم الطلب" + صفحة طلباتي

☒ التجربة: بتعمل طلب كامل ويبتسجل في الداتابيز.

8 لوحة الأدمن (Admin Panel)

☐ حماية مسارات الأدمن

☐ إدارة المنتجات (إضافة/تعديل/حذف)

☐ إدارة الطلبات (تغيير الحالة)

☒ التجربة: الأدمن يدير المتجر من الواجهة.

9 ✨ لمسات أخيرة (Polish & Extras)

- ☐ التقييمات (Reviews)
- ☐ تكامل دفع (Stripe/Paymob) أو دفع عند الاستلام
- ☐ Pagination + حالات التحميل والأخطاء
- ☐ تصميم متجاوب (Responsive)
- ☐ النشر: Backend على Render + Frontend على Vercel

▶ نبدأ منين؟

1. **ابدأ هنا** **Phase 0**: تتأكد إن Node و MongoDB جاهزين.
2. نعمل فولدر `backend` ونثبت الحزم.
3. نكتب `server.js` + `config/db.js` ونشغل السيرفر.
4. أول ما "MongoDB Connected" تظهر ← ننتقل لـ Phase 1 (الموديلز).
السبب إننا بنبدأ بالباك-إند: الفرونت مالوش لازمة من غير API بيرجع داتا.

🧪 أدوات هتفيدك

Postman — بديل لتجربة الـ API

Thunder Client — إكستنشن في VS Code
لتجربة الـ API

nodemon — يعيد تشغيل السيرفر تلقائيًا

MongoDB Compass — تشوف الداتا بعينك

خطة تنفيذ مشروع Amazon Clone · افتح الملف في المتصفح لقراءتها بشكل منظم